

blynt

ENGINEERING & OPERATIONS PLAYBOOK

Site Reliability Engineering • Operational Runbooks • Incident Response Protocols
Disaster Recovery Procedures • Financial Accounting Standard Operating Procedures
Authoritative Operations Manual • October 2026

Operational Governance Notice

This standalone Playbook prescribes mandatory standard operating procedures, incident triage protocols, disaster recovery runbooks, and financial period management rules for engineers, DevOps personnel, and financial administrators operating the blynt platform.

Document Ref: BLYNT-OPS-PLAYBOOK-V1
System Name: blynt (Internal Code: Ledgerline)
System Target Availability: 99.95% Monthly SLA
Financial Precision Rule: Zero-Drift Arbitrary Fixed-Point Invariant
Recovery Objectives: RPO = 0 (Zero Financial Data Loss) | RTO < 30 Minutes
Document Classification: Confidential Operational & Engineering Playbook
Authorized Audience: DevOps Engineers, SREs, Tech Leads, Financial Controllers

blynt — Engineering & Operations Playbook

Document Classification: Engineering Runbooks & Operational Governance

Target Systems: AWS Cloud Infrastructure (`eu-west-2` London), NestJS 11 Backend, PostgreSQL 16 Aurora/RDS, Next.js 15 Web Client

Version: 1.0 Production Readiness

Document Ref: `BLYNT-OPS-PLAYBOOK-V1`

Table of Contents

- **1. Operational Charter, SLOs & Incident Response Framework**
- 1.1 Service Level Objectives (SLOs) & Reliability Targets
- 1.2 Severity Classification Matrix (SEV-1 to SEV-4)
- 1.3 Incident Response Roles & Communication Escalation
- **2. Local Engineering & Development Runbook**
- 2.1 Developer Prerequisites & Tooling
- 2.2 Repository Architecture & Workspace Setup
- 2.3 Local Environment Configuration (.env files)
- 2.4 Database Provisioning, Prisma Migration & Seeding
- 2.5 Running Applications Locally (Backend, Frontend, Full Monorepo)
- 2.6 Running the Verification Test Suites (26 Suites / 127 Tests)
- **3. CI/CD, Deployment & Release Playbook**
- 3.1 Pull Request Quality Gate & Automated Testing
- 3.2 Staging & Production Deployment Pipeline
- 3.3 Zero-Downtime Database Schema Migration Strategy
- 3.4 Production Rollback Runbook
- **4. Database Administration & Maintenance Playbook**
- 4.1 Connection Pool Sizing & Management (PgBouncer & Prisma)
- 4.2 Point-In-Time Recovery (PITR) & Automated Snapshots
- 4.3 Immutable Audit Logs Health Verification (`trg_audit_logs_immutable`)
- 4.4 PostgreSQL `btree_gist` Temporal Constraints Index Maintenance
- **5. Financial Operations & Accounting Runbooks (Standard SOPs)**
- 5.1 Runbook FIN-01: Tenant Onboarding & Chart of Accounts Initialization
- 5.2 Runbook FIN-02: Month-End Close & Soft Lockdown Protocol
- 5.3 Runbook FIN-03: Statutory Year-End Close & Hard Lockdown Protocol
- 5.4 Runbook FIN-04: GAAP-Compliant Journal & Invoice Reversals
- 5.5 Runbook FIN-05: Accounts Receivable Reconciliation & Bad Debt Write-Off
- **6. Incident Response Runbooks (IR Protocols)**

- 6.1 Playbook IR-01: General Ledger Imbalance / Accounting Drift (SEV-1)
- 6.2 Playbook IR-02: Database Connection Exhaustion / Latency Spike (SEV-2)
- 6.3 Playbook IR-03: Multi-Tenant Boundary Isolation Breach Alert (SEV-1)
- 6.4 Playbook IR-04: Cloud Deployment Failure / Container CrashLoop (SEV-1)
- **7. Disaster Recovery & Regional Failover Playbook**
- 7.1 Disaster Recovery Objectives ($RPO = 0$, $RTO < 30$ mins)
- 7.2 Complete Region Outage Failover (`eu-west-2` to `eu-west-1`)
- 7.3 Snapshot Restoration & Forensic Integrity Validation
- **8. Security Operations, Key Rotation & Compliance Playbook**
- 8.1 Periodic Secrets Rotation (JWT, Database, Session Keys)
- 8.2 Making Tax Digital (MTD) & Statutory 6-Year Audit Trail Retention
- 8.3 Least Privilege IAM Policy Auditing
- **9. Monitoring, Observability & Alerting Reference**
- 9.1 Synthetic Health Probes (`/api/v1/health`)
- 9.2 Critical CloudWatch Alarms & Thresholds
- 9.3 Correlation ID Log Tracing (`x-request-id`) with CloudWatch Insights

1. Operational Charter, SLOs & Incident Response Framework

The blynt platform provides institutional-grade double-entry financial accounting. Operational failure carries direct statutory and fiduciary consequences for tenant businesses.

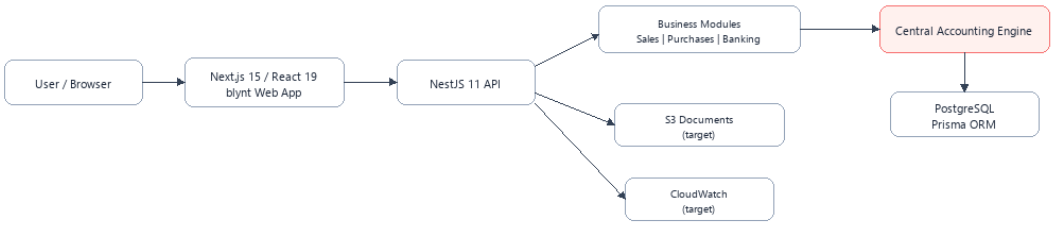


Figure 1 - High-level blynt system architecture

1.1 Service Level Objectives (SLOs) & Reliability Targets

Metric	Target SLO	Measurement Window	Error Budget
API Availability	**99.95%**	Monthly Rolling (30 days)	21.6 minutes downtime / month
P95 Request Latency	**< 150 ms**	5-minute rolling window	Standard read/write operations

P99 Request Latency	**< 250 ms**	5-minute rolling window	Complex reporting endpoints
Financial Data Loss (RPO)	**0 seconds ($RPO = 0$)**	Continuous	Multi-AZ synchronous commit
Recovery Time (RTO)	**< 30 minutes ($RTO < 30 ext{m}$)**	Disaster recovery event	Full cross-region failover
Accounting Invariant Drift	**0.0000 GBP (Zero drift)**	Continuous integrity scanner	0 balance discrepancies tolerated

1.2 Severity Classification Matrix

Severity Level	Definition	Impact Scope	Initial Response Target	Update Cadence
SEV-1 (Critical)	Core API unavailable, database down, trial balance out of equilibrium, or security breach.	All tenants or core financial integrity impaired.	**< 15 Minutes** (24/7 on-call pager)	Every 30 minutes
SEV-2 (Major)	Major subsystem failure (e.g. Invoicing disabled, reports failing), high latency (> 2s), or soft-lock errors.	Significant tenant cohort or major feature unusable.	**< 30 Minutes** (Business hours & on-call)	Every 2 hours
SEV-3 (Moderate)	Non-blocking bug, UI styling defect, AI document suggestion error with fallback available.	Single tenant or non-critical functionality.	**< 4 Hours** (Next business window)	Daily
SEV-4 (Minor)	Cosmetic defect, documentation update, feature request.	Minimal operational impact.	**< 2 Business Days**	Weekly sprint review

1.3 Incident Response Roles & Escalation Matrix

During an active SEV-1 or SEV-2 incident, an **Incident Command Team** is automatically activated:

```
[ Incident Commander (Lead SRE) ]
```

```
       │
       ├── [ Tech Lead / Forensic Engineer ]  (Identifies root cause, writes patch)
       ├── [ Operations / DevOps Lead ]        (Executes rollbacks, scales infrastructure)
       └── [ Communications Officer ]          (Maintains tenant status page, informs leadership)
```

1. **Incident Commander (IC):** Directs triage, authorizes emergency database mutations, assigns tasks, and ensures no uncoordinated actions take place.

2. **Technical Lead:** Investigates logs, correlates `x-request-id`, reproduces the failure in sandbox, and validates fix.

3. **Operations Lead:** Executes infrastructure remediation (container restart, scaling, failover, rollback).

4. **Communications Officer:** Publishes status updates to tenant administrators every 30 minutes until resolution.

2. Local Engineering & Development Runbook

2.1 Developer Prerequisites & Tooling

Engineers contributing to blynt must have the following toolchains installed:

- **Node.js:** `v22.x LTS` (Node 20+ supported; Node 22 recommended)
- **npm:** `v10.x` or later
- **PostgreSQL:** `v16.x` (or Docker running PostgreSQL 16)
- **Git:** `2.40+`
- **AWS CLI:** `v2.x` (for cloud deployment workflows)

2.2 Monorepo Layout

```
d:/Project/F!
├── apps/
│   ├── api/                # NestJS 11 Backend Service (Port 4000)
│   │   ├── prisma/         # schema.prisma, migrations, seeds
│   │   └── src/            # Domain modules, accounting engine, common
│   └── web/                # Next.js 15 Web Application (Port 3000)
│       └── src/            # App router, components, contexts, lib/api.ts
├── docs/                   # Technical & Operations Documentation
│   └── diagrams/           # Architecture & flowchart diagrams
└── scripts/                # Automation, migration & PDF generators
```

2.3 Local Environment Configuration

Backend Configuration (`apps/api/.env`):

```
PORT=4000
NODE_ENV=development
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/blynt_dev?schema=public
SESSION_SECRET=local-dev-session-secret-min-32-chars-long-12345
CORS_ORIGIN=http://localhost:3000
LOG_LEVEL=debug
```

Frontend Configuration (`apps/web/.env.local`):

```
NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1
PORT=3000
```

2.4 Database Provisioning, Migration & Seeding

Execute the following commands from the project root:

```
# 1. Install root and workspace dependencies
npm install

# 2. Change directory to api workspace
cd apps/api

# 3. Push Prisma schema to local PostgreSQL and create database
npx prisma db push

# 4. Generate Prisma Client bindings
npx prisma generate

# 5. Seed default nominal accounts, test organisation, and demo user
npm run db:seed
```

2.5 Running Applications Locally

Option A: Running Both Backend & Frontend Simultaneously (Root Monorepo)

```
# From workspace root:
npm run dev
```

Option B: Running Individual Services in Dedicated Terminals

```
# Terminal 1 - Backend API (NestJS 11 in watch mode):
cd apps/api
npm run start:dev
```

```
# Terminal 2 - Frontend Web Client (Next.js 15 Turbopack):
cd apps/web
npm run dev
```

- **Backend Health Check:** `curl http://localhost:4000/api/v1/health`
- **Frontend Web App:** Open `http://localhost:3000` in browser.

2.6 Running the Verification Test Suites

Before pushing any code, all 26 test suites (127 tests) must pass with zero failures:

```
# Run unit and service test suites across the API:
cd apps/api
npm run test

# Run tests with test coverage reporting:
npm run test:cov
```

- **Required Outcome:** `Test Suites: 26 passed, 26 total. Tests: 127 passed, 127 total.`

3. CI/CD, Deployment & Release Playbook

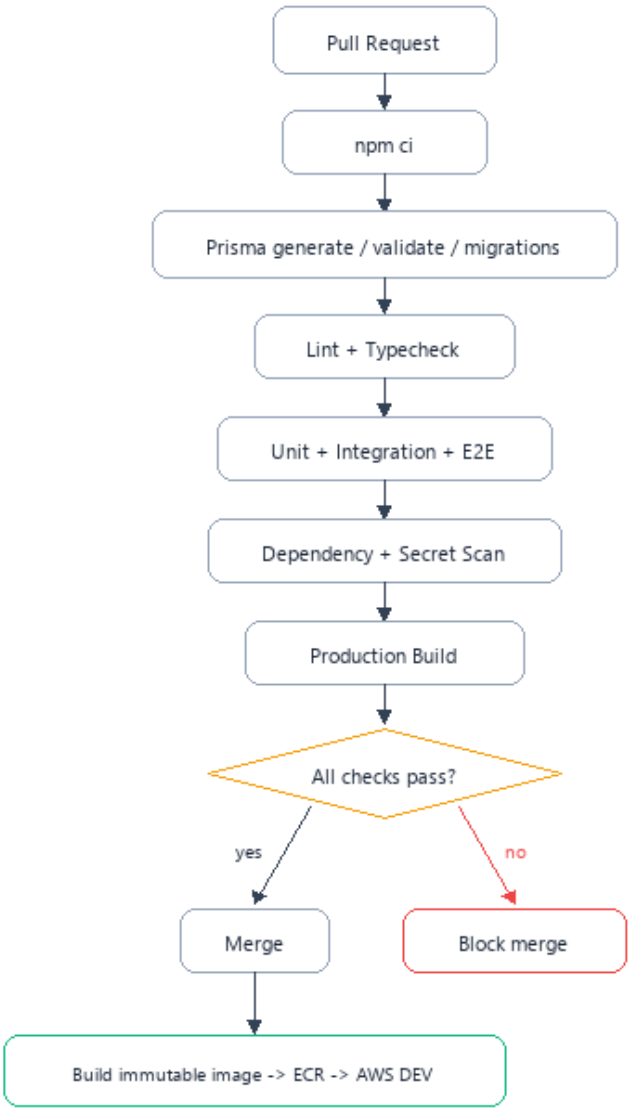


Figure 10 - Pull-request quality gate and AWS DEV deployment flow

3.1 Pull Request Quality Gate & Automated Testing

Every pull request to `main` must pass the following automated gate before code review:

1. **Dependency Audit:** `npm audit --omit=dev` verifies no high/critical vulnerabilities.
2. **Prisma Schema Validation:** `npx prisma validate` confirms database schema integrity.

3. **Type-Checking:** `npm run typecheck` across both `apps/api` and `apps/web`.
4. **Linting:** ESLint rules enforced with zero warnings.
5. **Active Unit Tests:** All 26 test suites (127 tests) must pass.
6. **Container Build Verification:** Docker multi-stage build produces runnable image.

3.2 Staging & Production Deployment Pipeline

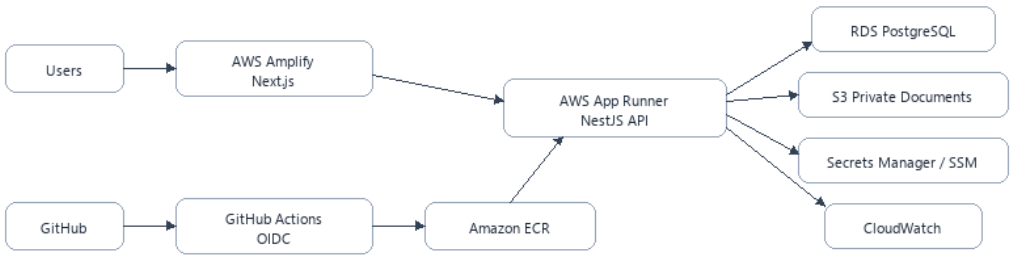


Figure 8 - Low-cost AWS DEV architecture in eu-west-2

Figure 8 - Low-cost AWS DEV architecture in eu-west-2

Automated Cloud Architecture in `eu-west-2` (London):

- **Web Tier:** AWS Amplify Hosting (SSR Next.js 15).
- **API Tier:** AWS App Runner containerized microservice running NestJS 11.
- **Database Tier:** Amazon RDS PostgreSQL 16 Multi-AZ instance.

Step-by-Step Deployment Procedure:

1. Merge approved Pull Request to `main`.
2. GitHub Actions builds production container image and pushes to Amazon ECR:

```
docker build -t blynt-api:latest -f apps/api/Dockerfile .
docker tag blynt-api:latest 123456789012.dkr.ecr.eu-west-2.amazonaws.com/blynt-api:release-v1.0.4
docker push 123456789012.dkr.ecr.eu-west-2.amazonaws.com/blynt-api:release-v1.0.4
```

3. Execute database migration job in Amazon ECS task or App Runner pre-deploy hook:

```
npx prisma migrate deploy
```

4. App Runner pulls new image tag and executes health check probe against `/api/v1/health`.
5. Traffic shifts smoothly with zero downtime.

3.3 Zero-Downtime Database Schema Migration Strategy

Financial databases cannot tolerate locks that stall transactions. Follow the **Expand/Contract Pattern**:

1. **Phase 1 (Expand):** Add new columns as optional/nullable. Add new indexes concurrently (`CREATE INDEX CONCURRENTLY`).

2. **Phase 2 (Dual-Write):** Deploy application code that reads from old column and writes to both old and new.

3. **Phase 3 (Backfill):** Backfill existing rows via background script in small batches (1000 rows/batch).

4. **Phase 4 (Switch):** Deploy code reading exclusively from the new column.

5. **Phase 5 (Contract):** Drop legacy column in a subsequent scheduled release.

3.4 Production Rollback Runbook

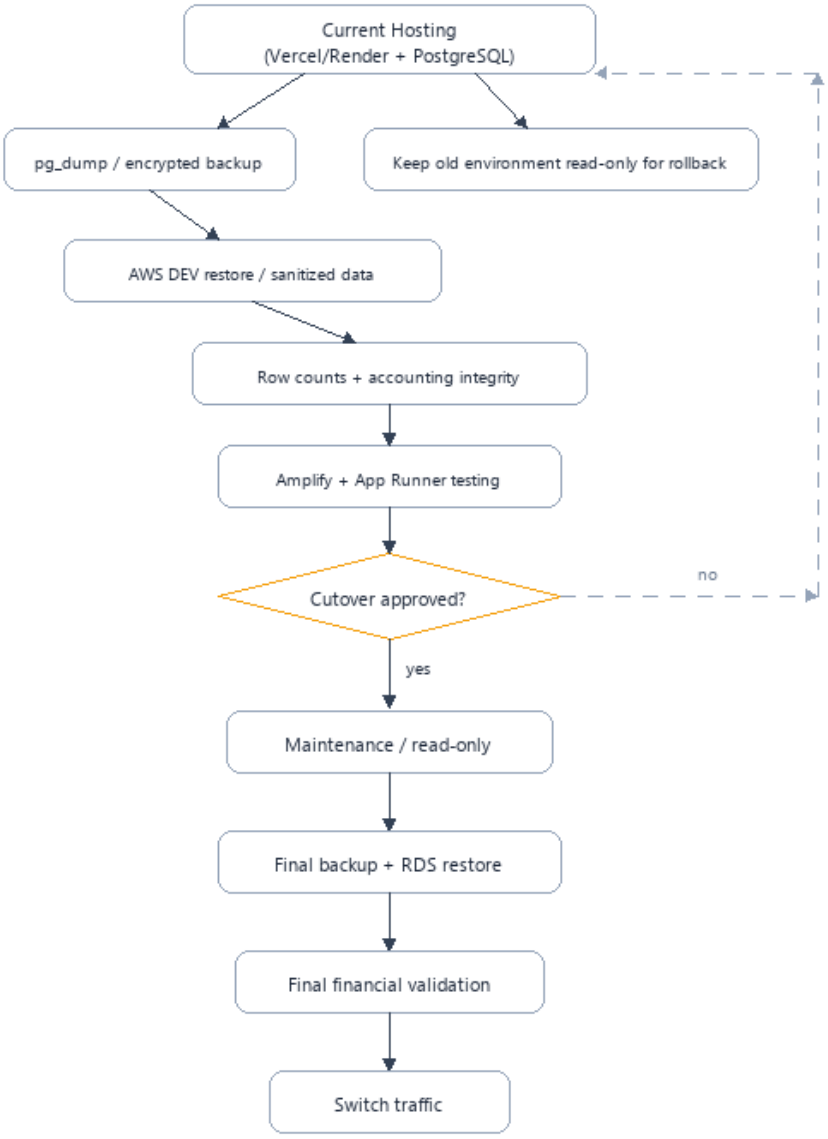


Figure 9 - Safe current-hosting to AWS migration and rollback path

If a release introduces unexpected behavior or errors:

Immediate Application Rollback:

```
# 1. Update AWS App Runner to previous stable image tag
aws apprunner start-deployment   --service-arn arn:aws:apprunner:eu-west-
2:123456789012:service/blynt-api-prod/abc123   --region eu-west-2

# 2. Revert AWS Amplify Frontend release
aws amplify start-job   --app-id d123456789   --branch-name main   --job-type RELEASE   --commit-id
<PREVIOUS_COMMIT_SHA>
```

4. Database Administration & Maintenance Playbook

4.1 Connection Pool Sizing & Management

Prisma Client manages an internal connection pool. To prevent PostgreSQL connection starvation:

- **Rule:** `Max Connections = (App Runner Container Instances * Pool Size) + Reserve Pool`.
- Configure in `DATABASE_URL`: `postgresql://...?schema=public&connection_limit=15&pool_timeout=10`.
- For production workloads exceeding 100 concurrent workers, connect through AWS RDS Proxy / PgBouncer.

4.2 Point-In-Time Recovery (PITR) & Automated Snapshots

- **Automated Daily Snapshots:** Scheduled at 02:00 UTC with 35-day retention.
- **Continuous Write-Ahead Log (WAL) Archival:** Backed up to Amazon S3 every 5 minutes.
- **Point-In-Time Recovery:** Enables database restoration to any second within the retention window.

Initiating a Test PITR Restore:

```
aws rds restore-db-instance-to-point-in-time   --source-db-instance-identifier blynt-rds-prod   --
target-db-instance-identifier blynt-rds-pitr-audit-test   --restore-time 2026-10-11T23:00:00Z   --
db-subnet-group-name blynt-private-subnets   --region eu-west-2
```

4.3 Immutable Audit Logs Health Verification

The PostgreSQL trigger `trg_audit_logs_immutable` must be verified monthly to ensure compliance:

```
-- Test query: Attempting to update an audit log row MUST fail:
UPDATE audit_logs
SET action = 'TAMPERED'
WHERE id = (SELECT id FROM audit_logs LIMIT 1);

-- Expected Result:
-- ERROR: Statutory Compliance Violation: audit_logs are immutable and cannot be updated or deleted
```

If this query succeeds, declare a **SEV-1 Security Incident** immediately.

4.4 PostgreSQL `btree_gist` Index Maintenance

Ensure temporal exclusion indexes on `financial_years` and `accounting_periods` are healthy:

```
-- Verify exclusion constraints are valid and not marked invalid:
SELECT conname, contype, convalidated
FROM pg_constraint
WHERE conname IN ('financial_years_no_overlap', 'accounting_periods_no_overlap');
```

5. Financial Operations & Accounting Runbooks (Standard SOPs)

5.1 Runbook FIN-01: Tenant Onboarding & Chart of Accounts Initialization

Objective: Provision a new enterprise organisation with verified nominal accounts.

```
Step 1: Create Organization Record (legalName, slug, baseCurrency = 'GBP')
Step 2: Seed Standard Chart of Accounts (Nominal ranges 1000 - 8999)
Step 3: Mark Control Accounts (1100 AR, 2000 AP, 2100 VAT, 3100 Retained Earnings)
Step 4: Create Initial Financial Year (12 monthly periods generated)
Step 5: Assign User as OWNER role in organization_members
Step 6: Emit AuditLog (EVENT: ORGANIZATION_INITIALIZED)
```

Verification Query:

```
SELECT code, name, account_type, is_control_account
FROM accounts
WHERE organization_id = '<ORG_ID>'
ORDER BY code ASC;
-- Must return all 25+ base nominal accounts.
```

5.2 Runbook FIN-02: Month-End Close & Soft Lockdown Protocol

Objective: Lock the completed accounting period to prevent standard user modifications while allowing final accountant review.

1. **Pre-Close Reconciliation:**

- Scan for unposted draft journals:

```
SELECT COUNT(*) FROM journal_entries
WHERE organization_id = '<ORG_ID>' AND status = 'DRAFT'
AND posting_date <= '<PERIOD_END_DATE>';
```

Must be 0.

- Scan for draft sales invoices:

```
    SELECT COUNT(*) FROM invoices
    WHERE organization_id = '<ORG_ID>' AND status = 'DRAFT'
    AND issue_date <= '<PERIOD_END_DATE>';
```

Must be 0.

2. **Execute Soft Lockdown via API:**

```
   curl -X POST https://api.blynt.io/api/v1/organizations/<ORG_ID>/periods/<PERIOD_ID>/soft-lock
-H "Authorization: Bearer <ADMIN_TOKEN>"
```

3. **Verify Trial Balance Equilibrium:** Generate Trial Balance report; confirm `isBalanced: true` and `difference: 0.00`.

5.3 Runbook FIN-03: Statutory Year-End Close & Hard Lockdown Protocol

Objective: Permanently close the financial year, transfer net income to retained earnings, and lock all 12 periods into an immutable state.

```
1. Calculate Net Profit/Loss:
   Net Income = Total Credit (4000-4999) - Total Debit (5000-8999)
2. Generate Year-End Closing Journal:
   - Debit all Revenue Accounts (clearing to zero)
   - Credit all Expense Accounts (clearing to zero)
   - Credit/Debit Retained Earnings (Nominal 3100) with Net Income
3. Post Year-End Closing Journal (Status: POSTED)
4. Execute Hard Lockdown on Financial Year:
   UPDATE financial_years SET status = 'HARD_LOCKED' WHERE id = '<FY_ID>';
   UPDATE accounting_periods SET status = 'HARD_LOCKED' WHERE financial_year_id = '<FY_ID>';
```

5.4 Runbook FIN-04: GAAP-Compliant Journal & Invoice Reversals

Objective: Correct an erroneous posted transaction without mutating the historical audit trail.

1. Identify the posted journal ID (`journalId`).

2. Invoke `JournalReversalService.reverseJournal`:

- System generates counter-journal swapping debits and credits.
- Assigns sequence number `JE-YYYY-XXXXXX`.
- Sets original journal status to `REVERSED` and links `reversedByJournalId`.

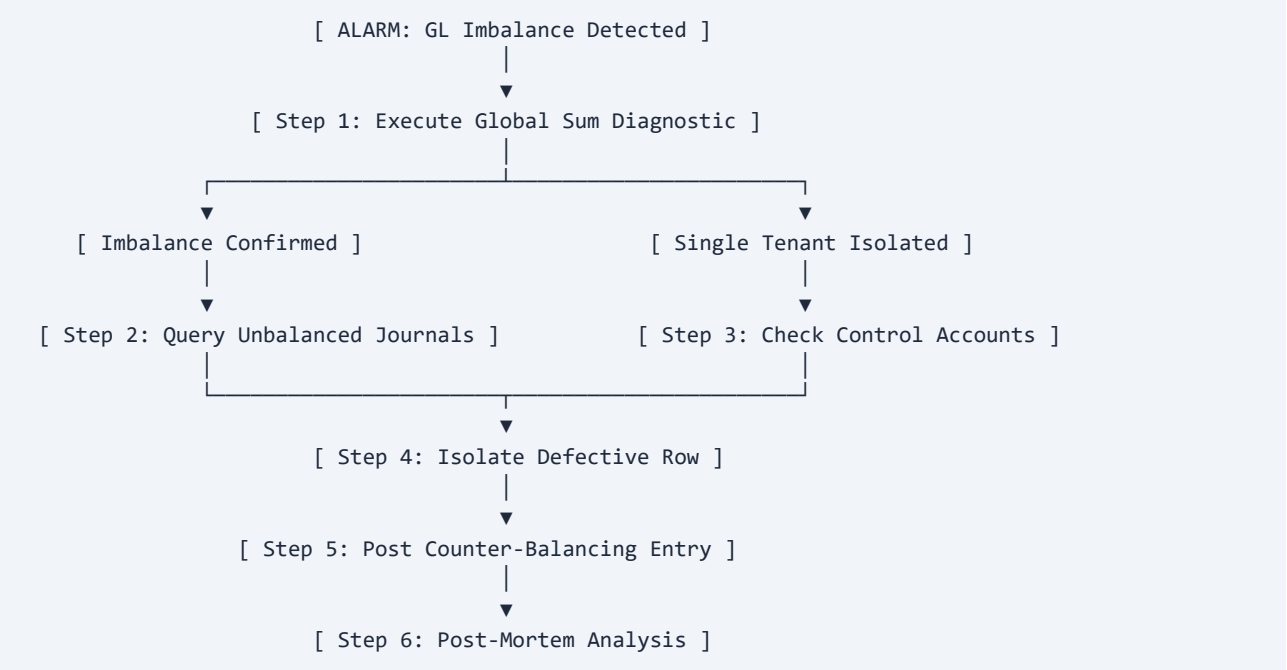
3. If correcting an invoice:

- Reversal journal is generated automatically.
- Invoice status transitions to `VOIDED`.
- Linked journal reference remains intact for statutory traceability.

6. Incident Response Runbooks (IR Protocols)

6.1 Playbook IR-01: General Ledger Imbalance / Accounting Drift (SEV-1)

Trigger: `AccountingIntegrityService` reports `isBalanced: false` or CloudWatch alert `ALARM_GL_IMBALANCE`.



Forensic Diagnostic Query:

```
-- Find any journal where sum of debits does not equal sum of credits:
SELECT
  j.id,
  j.organization_id,
  j.journal_number,
  j.status,
  SUM(l.debit) AS total_debit,
  SUM(l.credit) AS total_credit,
  ABS(SUM(l.debit) - SUM(l.credit)) AS diff
FROM journal_entries j
JOIN journal_lines l ON l.journal_entry_id = j.id
WHERE j.status = 'POSTED'
GROUP BY j.id, j.organization_id, j.journal_number, j.status
HAVING SUM(l.debit) <> SUM(l.credit);
```

Remediation Protocol:

1. If defective journal identified: Check `audit_logs` for user or system process that executed the write.

2. Verify why validation invariant did not catch the error.

3. Post a GAAP balancing adjustment journal approved by Lead Accountant and Incident Commander.

4. **Never directly mutate `journal_lines` in SQL.** Always post through the accounting engine.

6.2 Playbook IR-02: Database Connection Exhaustion / Latency Spike (SEV-2)

Trigger: CloudWatch alert `RDS_DatabaseConnections > 80%` or HTTP 504 Gateway Timeouts.

Immediate Triage:

```
-- 1. Identify active running queries and blocking locks:
SELECT pid, age(clock_timestamp(), query_start), usename, state, query
FROM pg_stat_activity
WHERE state != 'idle' AND query NOT ILIKE '%pg_stat_activity%'
ORDER BY query_start ASC;

-- 2. Terminate long-running blocking query (replace <PID>):
SELECT pg_terminate_backend(<PID>);
```

Preventive Actions:

- Check for missing index on `journal_lines (organization_id, account_id)`.
- Restart App Runner service to cycle stale connection pools.
- Temporarily increase RDS instance class if genuine transaction volume spike.

6.3 Playbook IR-03: Multi-Tenant Boundary Isolation Breach Alert (SEV-1)

Trigger: Any query attempt where `organization_id` does not match the authenticated session's tenant scope.

```
-- 1. Immediate containment: Revoke active sessions for the compromised user:
DELETE FROM user_sessions WHERE user_id = '<COMPROMISED_USER_ID>';

-- 2. Audit forensics: Search audit_logs for any mutations performed by this actor:
SELECT created_at, action, entity_type, entity_id, ip_address, request_id
FROM audit_logs
WHERE user_id = '<COMPROMISED_USER_ID>'
ORDER BY created_at DESC;
```

7. Disaster Recovery & Regional Failover Playbook

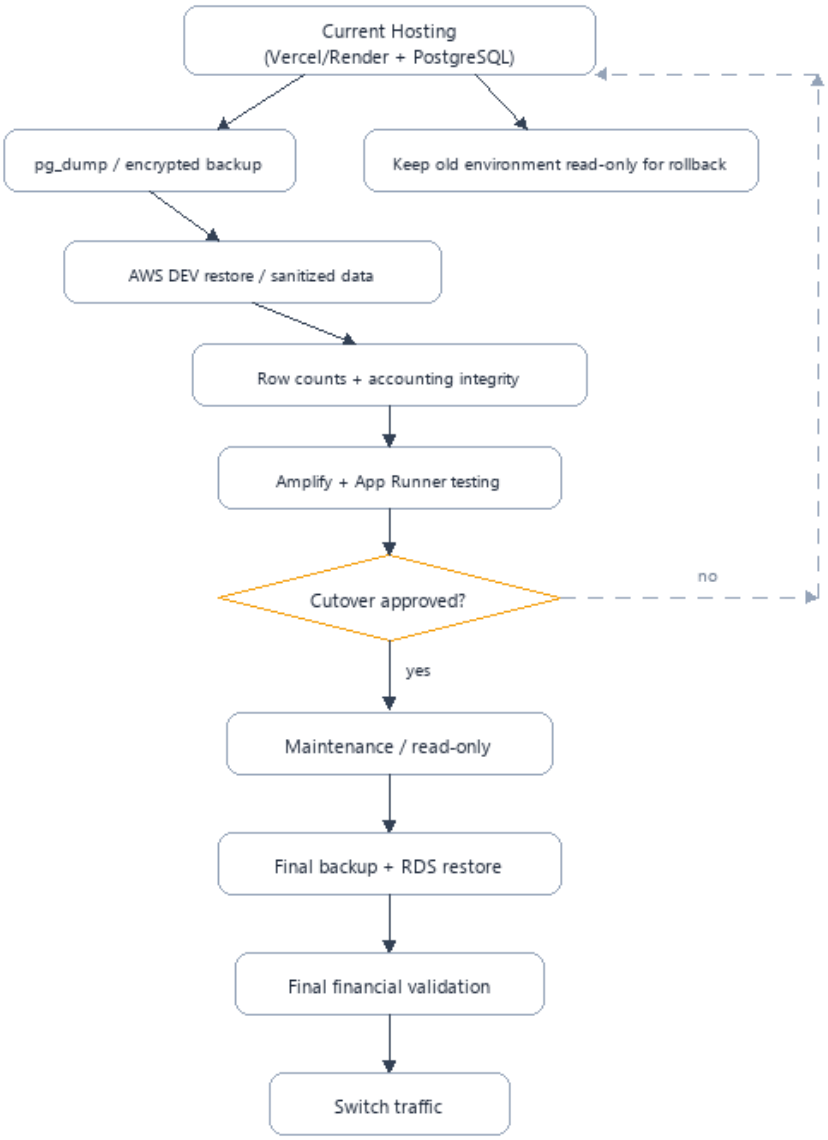


Figure 9 - Safe migration and rollback path

7.1 Disaster Recovery Objectives

- **Recovery Point Objective (RPO): 0 seconds** (synchronous Multi-AZ standby).
- **Recovery Time Objective (RTO): < 30 minutes** (full failover to standby region).

7.2 Complete Region Outage Failover (`eu-west-2` to `eu-west-1`)

If AWS London experiences catastrophic failure:

1. **Promote Cross-Region Read Replica in Ireland (`eu-west-1`):**

```
   aws rds promote-read-replica      --db-instance-identifier blynt-rds-dr-ireland      --region eu-
west-1
```

2. **Deploy Backend App Runner Service in Ireland:**

```
   aws apprunner create-service      --service-name blynt-api-dr      --source-configuration
ImageRepository={ImageIdentifier="...",ImageRepositoryType="ECR"}      --region eu-west-1
```

3. **Update Route 53 DNS Record:**

```
   aws route53 change-resource-record-sets      --hosted-zone-id Z1234567890      --change-batch
file://scripts/aws/failover-dns-ireland.json
```

4. **Execute Post-Failover Accounting Verification:**

- Run `AccountingIntegrityService.checkIntegrity()` across all active tenants.
- Verify zero records lost.

8. Security Operations, Key Rotation & Compliance Playbook

8.1 Periodic Secrets Rotation

Execute every 90 days for production security hygiene:

```
# 1. Generate new 64-character session secret:
NEW_SECRET=$(openssl rand -hex 32)

# 2. Update AWS Secrets Manager:
aws secretsmanager update-secret   --secret-id blynt/prod/session-secret   --secret-string
"{"SESSION_SECRET":"$NEW_SECRET"}"   --region eu-west-2
```

8.2 Making Tax Digital (MTD) & Statutory 6-Year Retention

Under UK statutory law (Companies Act 2006 & HMRC VAT Notice 700/21), accounting records and audit trails must be retained for **6 years plus current financial year**:

- All S3 automated database backup buckets are configured with **AWS S3 Object Lock in Compliance Mode** (WORM: Write Once, Read Many).
- No administrator or IAM role (including root) can delete backups within the retention window.

9. Monitoring, Observability & Alerting Reference

9.1 Synthetic Health Probes

External uptime monitors (e.g., AWS Route 53 Health Checks / Better Uptime) query every 30 seconds:

```
GET /api/v1/health HTTP/1.1
Host: api.blynt.io
```

Expected 200 OK Response:

```
{
  "status": "ok",
  "info": {
    "database": { "status": "up" },
    "memory": { "status": "up" },
    "uptime": 86420
  }
}
```

9.2 Critical CloudWatch Alarms

Alarm Name	Metric / Condition	Action	Notification Target
`ALARM_API_5XX_RATE`	`5XX Error Rate > 1%` over 5 min	PagerDuty SEV-2 Alert	SRE On-Call Pager
`ALARM_GL_IMBALANCE`	`Metric: BalanceDiscrepancy > 0`	Immediate PagerDuty SEV-1	Tech Lead & Finance Controller
`ALARM_RDS_CPU`	`CPUUtilization > 85%` for 10 min	Auto-scale / Notify Ops	DevOps Slack `#ops-alerts`
`ALARM_RDS_STORAGE`	`FreeStorageSpace < 20 GB`	Auto-expand storage	DevOps Slack `#ops-alerts`

9.3 Correlation ID Log Tracing (`x-request-id`)

Every incoming HTTP request is assigned a unique `x-request-id`. To trace an incident across CloudWatch Logs:

```
fields @timestamp, @message
| filter requestId = '9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d'
| sort @timestamp asc
| limit 100
```

END OF BLYNT ENGINEERING & OPERATIONS PLAYBOOK